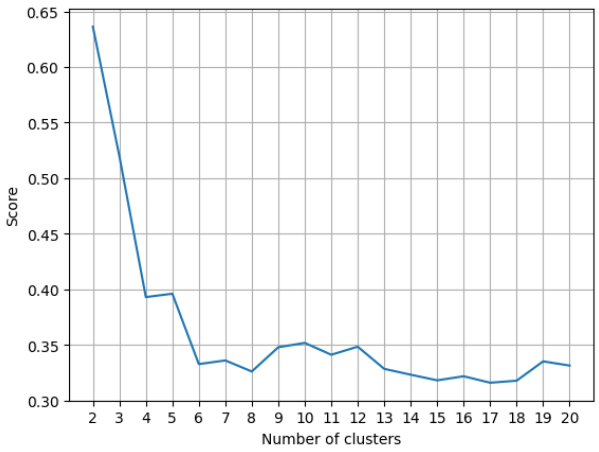
Zero Edoardo, SMID

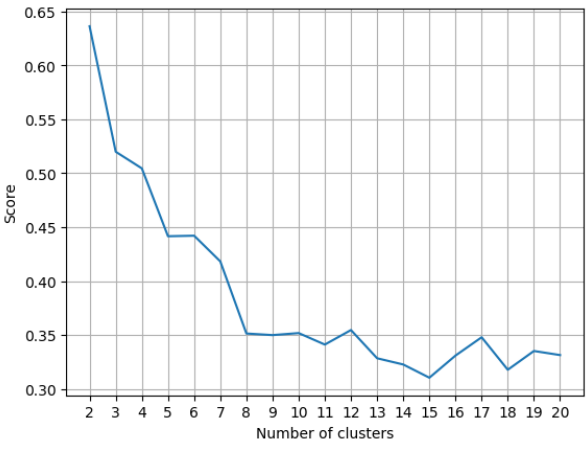
Laboratorio 7 : Algoritmo K-Means

In questo laboratorio è stata studiato l'approccio di clustering *K-means* , implementato con l'algoritmo di *Lloyd*, utilizzato nei casi di apprendimento non supervisionato.
Sono state definite una funzione per generare dati sintetici e una che per calcolare le distanze tra due insiemi di punti. La funzione *lloyd* implementate accetta in input il training set X, i centri iniziali dei cluster e un numero intero che indica il numero massimo di iterazioni che l'algoritmo può compiere. L'algoritmo procede iterativamente assegnando ogni punto al cluster con il centroide più vicino e aggiornando i centri come media dei punti assegnati al cluster, se un cluster risulta vuoto, il suo centroide sarà aggiornato con un vettore generato causalmente , dove ogni coordinata è un valore uniforme nell'intervallo (-1,1).
L'algoritmo ideato in questa maniera termina dopo un numero di iterazioni stabilite inizialmente un'alternativa per non fermare l'algoritmo a priori potrebbe essere quella di fermarlo quando i centri aggiornati sono gli stessi dell'iterazione precedente raggiungendo , quindi, una convergenza.

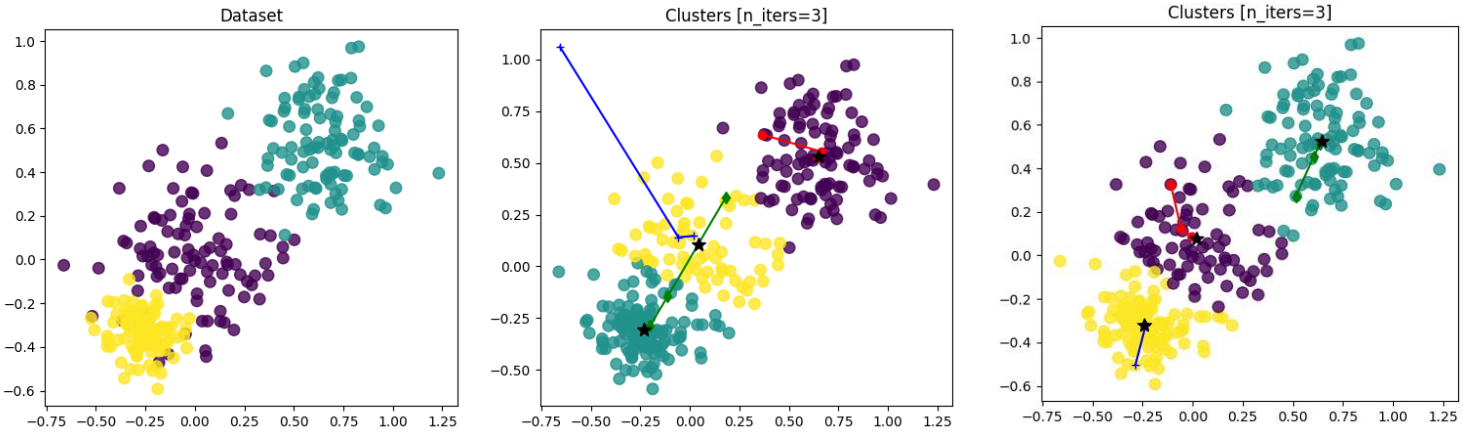
Si è proseguito provando questo algoritmo, i centri sono stati scelti randomicamente tenendo conto dei limiti di X e il numero di iterazioni è stato posto uguale al numero di cluster che si volevano realizzare.
Chiaramente una volta fissato il numero di centri e fatto girare il programma più volte i cluster finali erano diversi ogni volta , in quanto le distanze tra i punti e i centri random sono diversi ogni volta e quindi portano risultati differenti.
Se invece si sceglie un numero di cluster minore rispetto a quello con cui sono stati generati i dati alcune classi verranno raggruppate anche se potenzialmente potrebbero essere molto diverse, in caso invece si scelga un numero di cluster maggiore allora può succedere che una o più classi possano essere divise per ottenere più cluster. La scelta del numero di cluster è fondamentale e non scegliendo fedelmente il numero di cluster è probabile ottenere classi che non sono conformi alla struttura dei dati.

In seguito è stata utilizzata la libreria *Scikit-learn* per importare l'algoritmo *K-means* , la funzione di questa libreria possiede diverse modalità ad esempio è possibile scegliere quale opzione utilizzare per inizializzare i centri dei punti (*init='random' o 'k-means++'*) e si può anche impostare un numero *n_init* che corrisponde al numero di volte che l'algoritmo si ripete cambiando ogni volta il *seed* dei centroidi e infine selezionando il miglior risultato tra tutti.





A questo punto si è utilizzata la funzione *k-means* importata dalla libreria e si è provata su un numero diverso di cluster, calcolando uno score. Entrambi i grafici riportano la procedura con l'inizializzazione random dei centri con la differenza che a quello a sinistra è stato utilizzato *n_init=1* e quello a destra *n_init=10*, come si può vedere nel caso in cui il numero di ripetizioni era maggiore si sono ottenuti score più alti per tutte le grandezze dei cluster minori di 10 , tuttavia entrambi consigliano di utilizzare *K=2* cluster in quanto ha riportato uno score più elevato, ma anche tre è una buona scelta.
L'utilizzo dell'inizializzazione randomica può portare a una convergenza che può essere *sub-ottimale* , come è stato detto precedentemente può anche essere utilizzata la tecnica del *K-means++* per l'inizializzazione dei centri.
L'algoritmo funziona in questo modo: si sceglie casualmente il primo centro, poi per ogni punto si mantiene la distanza dal centro più vicino fino a quel momento. Si campiona un nuovo centro con probabilità proporzionale al quadrato della distanza e si ripete finché non si sono scelti k centri.
Ora, è stato preso un set di dati in cui *K-means* inizializzato in modo casuale ha difficoltà e si applica K-means++ per l'inizializzazione centrale, quindi si verifica se la nuova strategia di inizializzazione migliora il clustering.



A sinistra è riportato il dataset creato tramite la funzione *MixGauss* , al centro si trova la classificazione ottenuta tramite *K-Means* e infine a destra si trova quella ottenuta tramite *K-Means++*.
In entrambi i casi si può osservare che la posizione dei centri finali è la stessa ma cambia considerevolmente l'aggiornamento dei centri, in particolar modo la scelta iniziale , come spiegato precedentemente l'opzione di scegliere i centri casualmente può portare confusione nella classificazione dei cluster, in particolare se vengono scelti centri iniziali molto distanti dai dati è possibile che l'algoritmo confonda le classi.
In generale osservando i risultati si può concludere che la classificazione prodotta dal *K-means++* lavora più correttamente e porta risultati migliori, ovviamente commette comunque un errore perché ,ad esempio, i punti viola che si sovrappongono con la classe di punti gialla , vengono raggruppati nella classe gialla in quanto più vicini al centro del cluster giallo.